SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – DEBUGGING & OPTIMISATION TASK

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	**Assessment:**	Assessment Tool 3 – Debugging & Optimisation Task
Weightage:	10% of CIA	**Total Marks:**	100
CO2 Statement:	*Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)*		
Student Name:	R SARVESH	**Reg. No.:**	192521143
Section / Batch:	B.TECH IT	**Date:**	27/7/2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given program/code segment before identifying errors.
- Clearly identify logical, syntactical, and performance-related issues in the code.
- Apply systematic debugging techniques to correct the errors effectively.
- Optimize the given solution by improving efficiency, reducing unnecessary computations, or enhancing time complexity wherever possible.
- Provide proper justification for all corrections and optimization decisions.
- Write clean, structured, and well-documented code with appropriate comments.
- Maintain clarity, logical reasoning, and technical accuracy throughout the solution.

Question Type	Focus Area	Questions
Debugging & Optimisation Task	Identify errors, debug programs, and optimize algorithm efficiency using appropriate techniques	Q1

SECTION A – DEBUGGING & OPTIMISATION TASK

1. Debugging Binary Search Code (Incorrect Midpoint Formula with Bias)
The following Binary Search implementation computes the midpoint using a biased formula that can skip important candidates and distort the search interval for some inputs. Carefully inspect the midpoint calculation and explain why the search space is not divided correctly. Debug the algorithm to use a proper midpoint computation and consistent boundary updates. Optimize the implementation for readability and accurate handling of all edge cases. Analyze the corrected algorithm in best, average, and worst cases and rewrite the final version with proper documentation.

```
def binary_search(arr, key):
  low, high = 0, len(arr)-1
  while low <= high:
     mid = (low + high + 1) // 2 + 1   # ERROR
     if arr[mid] == key:
        return mid
     elif arr[mid] < key:
        low = mid + 1
     else:
        high = mid - 1
  return -1
```

Code of Conduct

I R SARVESH (192521143) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: R SARVESH

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%				
Debugging	25%				
Optimization	20%				
Analysis	20%				
Code Quality	15%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

SIMATS ENGINEERING
CSA0620-DESIGN AND ANALYSIS OF ALGORITHM

NAME: R SARVESH
REG. NO: 192521143
DEPT.: B.TECH INFORMATION TECHNOLOGY

CO2 ASSESSMENT TOOL 3

---

1. Debugging Binary Search Code (Incorrect Midpoint Formula with Bias)

   The following Binary Search implementation computes the midpoint using a biased formula that can skip important candidates and distort the search interval for some inputs. Carefully inspect the midpoint calculation and explain why the search space is not divided correctly. Debug the algorithm to use a proper midpoint computation and consistent boundary updates. Optimize the implementation for readability and accurate handling of all edge cases. Analyze the corrected algorithm in best, average, and worst cases and rewrite the final version with proper documentation.

   def binary_search(arr, key):
   low, high = 0, len(arr)-1
   while low <= high:
   mid = (low + high + 1) // 2 + 1 # ERROR
   if arr[mid] == key:
   return mid
   elif arr[mid] < key:
   low = mid + 1
   else:
   high = mid - 1
   return -1

   Problem Analysis
   The original midpoint calculation:
   mid = (low + high + 1) // 2 + 1
   is incorrect because:

- The extra +1 after division shifts the midpoint unnecessarily.
- It can make mid exceed the valid array index range.
- The search interval is not divided evenly, causing some candidates to be skipped.
- Incorrect midpoint selection may lead to wrong results or index errors.

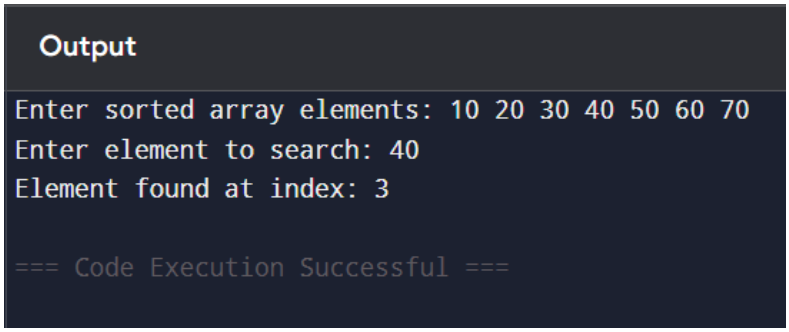
   Correct Midpoint Formula:
   mid = low + (high - low) // 2

This avoids overflow and correctly divides the search space.

Corrected Binary Search Algorithm (Python)

```
def binary_search(arr, key):

    low = 0
    high = len(arr) - 1

    while low <= high:

        mid = low + (high - low) // 2

        if arr[mid] == key:
            return mid

        elif arr[mid] < key:
            low = mid + 1

        else:
            high = mid - 1

    return -1

arr = list(map(int, input("Enter sorted array elements: ").split()))
key = int(input("Enter element to search: "))

result = binary_search(arr, key)

if result != -1:
    print("Element found at index:", result)
else:
    print("Element not found")
```

OUTPUT:



Complexity Analysis

Case	Time Complexity
Best Case	O(1)
Average Case	O(log n)
Worst Case	O(log n)

Space Complexity: O(1)

2. Debugging Maximum Element Search Code (Array Bounds Not Validated Before Access in C) The following maximum-element search function in C accesses arr[0] without first checking whether the pointer is valid or whether the size is positive, which can lead to undefined behavior. Carefully analyze why low-level functions in C must validate both pointer and size assumptions before reading memory. Apply systematic debugging so that invalid input is handled safely and the maximum search proceeds only when the array is usable. Optimize the corrected implementation for defensive programming and clear return conventions. Analyze the time complexity of the corrected solution and rewrite the final C code with suitable comments.

```
int findMax(int arr[], int n) {
int i, maxVal = arr[0]; /* ISSUE */
for (i = 1; i &lt; n; i++) {
if (arr[i] &gt; maxVal)
maxVal = arr[i];
}
return maxVal;
}
```

Problem Analysis

The original code:
int maxVal = arr[0];
causes undefined behavior because:

- The pointer arr may be NULL.
- The array size n may be zero or negative.
- Accessing arr[0] without validation may cause memory errors.

Defensive Programming Solution:

Before accessing the array:

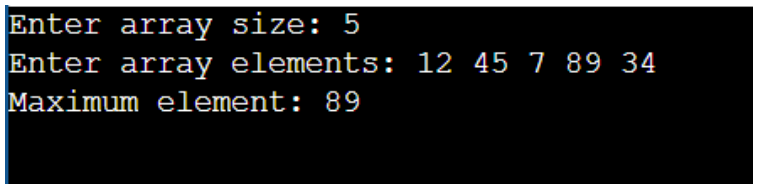
- Check whether arr == NULL.
- Check whether n <= 0.
- Return an appropriate error value when input is invalid.

Corrected C Program

```
#include <stdio.h>
#include <limits.h>
/*
   Finds maximum element in an integer array.

   Parameters:
      arr : integer array
      n   : size of array

   Returns:
      Maximum element if valid array,
      INT_MIN for invalid input
*/
int findMax(int arr[], int n)
{
   // Validate pointer and size
   if (arr == NULL || n <= 0)
   {
      return INT_MIN;
   }

   int maxVal = arr[0];

   for (int i = 1; i < n; i++)
   {
      if (arr[i] > maxVal)
      {
         maxVal = arr[i];
      }
   }

   return maxVal;
}
int main()
{
   int n;

   printf("Enter array size: ");
   scanf("%d", &n);

   if (n <= 0)
   {
      printf("Invalid array size\n");
      return 0;
```

```
    }

    int arr[n];

    printf("Enter array elements: ");

    for (int i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    int result = findMax(arr, n);

    if (result == INT_MIN)
    {
        printf("Invalid array\n");
    }
    else
    {
        printf("Maximum element: %d\n", result);
    }

    return 0;
}
```

OUTPUT:



Complexity Analysis

Case	Time Complexity
Best Case	O(n)
Average Case	O(n)
Worst Case	O(n)

Reason:

Every element must be checked once to guarantee finding the maximum value.

Space Complexity: O(1) (excluding input storage).